

Demoscene AI 2022 Demo - One-Page Summary

Generated from repository evidence on 2026-04-08.

What it is

A single-page, music-synced demoscene web app built with Vite + TypeScript, Canvas 2D, and Web Audio API. It plays a curated timeline of visual effects, transitions, and text cues over a soundtrack.

Who it is for

Primary persona: a demo creator/editor who wants to author or tweak audio-timed effect sections for a browser-based show.

What it does

- Runs a full-screen intro + section timeline with per-section effects, transitions, layering, and text cues.
- Synchronizes visuals to audio features (RMS/bass/mid/treble/beat) and supports manual beat injection via canvas taps.
- Supports many built-in effects plus WebGL2 effects with automatic Canvas2D fallback when WebGL2 is unavailable.
- Provides debug/editor controls, quality presets, framing modes, and URL-based rendering/performance tuning.
- Tracks live view counts using `/api/views` with GET/POST handling and KV-backed persistence when configured.
- Includes community workflows for submitting doodles and AI-generated effect ideas with moderation/review endpoints.

How it works (repo-evidenced architecture)

- Frontend app: `src/main.ts` wires canvas/UI, loads timeline config, initializes `AudioPlayer`, `Timeline`, and `Renderer`.
- Config layer: `src/config/loadConfig.ts` validates/normalizes raw JSON (audio, intro, sections, layers, transitions, cues).
- Playback core: `src/timeline/timeline.ts` computes intro/section mode, active transition windows, and active text cues by time.
- Audio analysis: `src/audio/audioPlayer.ts` uses Web Audio analyser data to emit beat + band energy features each frame.
- Rendering: `src/renderer/renderer.ts` and effect registry modules render the selected effect stack and transition blending.
- Service endpoints: `api/views.ts`, `api/doodles.ts`, `api/effects.ts` expose JSON APIs; they use `createKvClients()` for persistence.
- Data flow: `timeline.release.json` + `song.mp3` -> load/normalize -> frame loop reads audio+time -> timeline state -> renderer output

How to run (minimal)

- 1) Install Node.js 18+.
- 2) Run: `npm install`
- 3) Run: `npm run dev`
- 4) Open the local Vite URL and press Start.
- 5) Optional: replace `public/song.mp3` and edit `public/timeline.release.json` for custom timing/effects.